

Batch: 1

Roll No.:

16010123325

Experiment / Assignment / Tutorial No._____1

Title: Data Preprocessing and feature engineering in machine learning.

Aim: To understand & implement various data preprocessing and feature engineering techniques using Python for machine learning applications.

Expected Outcome of Experiment:

CO1: To establish a strong foundation in machine learning fundamentals.

Books/ Journals/ Websites referred:

1. <https://www.python.org/about/>
 2. <https://www.upgrad.com/blog/data-preprocessing-in-machine-learning/>
 3. <https://towardsdatascience.com/data-preprocessing-concepts-fa946d11c825>
 4. Data Mining: Concepts and Techniques by Jiawei Han, Micheline Kamber, Jian Pei
-

Introduction

What is a feature and why we need the engineering of it? Basically, all machine learning algorithms use some input data to create outputs. This input data comprise features, which are usually in the form of structured columns. Algorithms require features with some specific characteristic to work properly. Here, the need for feature engineering arises. Feature engineering efforts mainly have two goals:

- Preparing the proper input dataset which is compatible with the machine learning algorithm requirements.
- Improving the performance of machine learning models.

Different Preprocessing Techniques:

1. Imputation
2. Handling Outliers
3. Binning
4. Log Transform
5. One-Hot Encoding
6. Grouping Operations
7. Feature Split
8. Scaling

**** For a given dataset (housing.csv), implement the above preprocessing techniques, explain & attach the necessary snapshots for function/library used.**

1. Imputation

Imputing is the process of filling in missing data in a dataset. There are several techniques for imputing missing values in a dataset, such as mean imputation, median imputation, mode imputation, and KNN imputation.

```
# Importing Dataset
df=pd.read_csv('housing.csv')
print(df.shape)

df.info()
df.describe()
```

```
Out[1]:
(5000, 7)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 7 columns):
 #   Column              Non-Null Count  Dtype  
---  --
 0   Avg. Area Income    4998 non-null   object  
 1   Avg. Area House Age  4997 non-null   float64  
 2   Avg. Area Number of Rooms  4997 non-null  float64  
 3   Avg. Area Number of Bedrooms  4997 non-null  float64  
 4   Area Population      4996 non-null   float64  
 5   Price               4997 non-null   float64  
 6   Address             4997 non-null   object  
dtypes: float64(5), object(2)
memory usage: 273.6+ KB
```

	Avg. Area House Age	Avg. Area Number of Rooms	Avg. Area Number of Bedrooms	Area Population	Price
count	4997.000000	4997.000000	4997.000000	4996.000000	4.997000e+03
mean	5.977208	6.987482	3.981947	36170.033872	1.230331e+06
std	0.991680	1.005757	1.234117	8925.345004	3.532181e+05
min	2.644304	3.236194	2.000000	172.610686	1.593866e+04
25%	5.322270	6.298866	3.140000	29409.145500	9.974525e+05
50%	5.968905	7.002864	4.050000	36202.541765	1.232209e+06
75%	6.650994	7.665643	4.480000	42874.143687	1.471747e+06
max	9.519088	10.799588	6.500000	69821.713380	2.489066e+06

```

1. Imputation - is the process of handling null values

# Finding null values
df.isnull().sum()

...
Avg. Area Income      0
Avg. Area House Age   0
Avg. Area Number of Rooms  0
Avg. Area Number of Bedrooms  0
Area Population        0
Price                  0
Address                0
dtype: int64

# Median Imputation -> population
# Mode Imputation -> price
mean_population=df['Area Population'].median()
mean_price=df['Price'].mode()
mode_add=df['Address'].mode()

df['Area Population'] = df['Area Population'].fillna(mean_income)
df['Price'] = df['Price'].fillna(mean_price)
df['Address'] = df['Address'].fillna(mode_add)

df.isnull().sum()

...
Avg. Area Income      0
Avg. Area House Age   0
Avg. Area Number of Rooms  0
Avg. Area Number of Bedrooms  0
Area Population        0
Price                  0
Address                0
dtype: int64

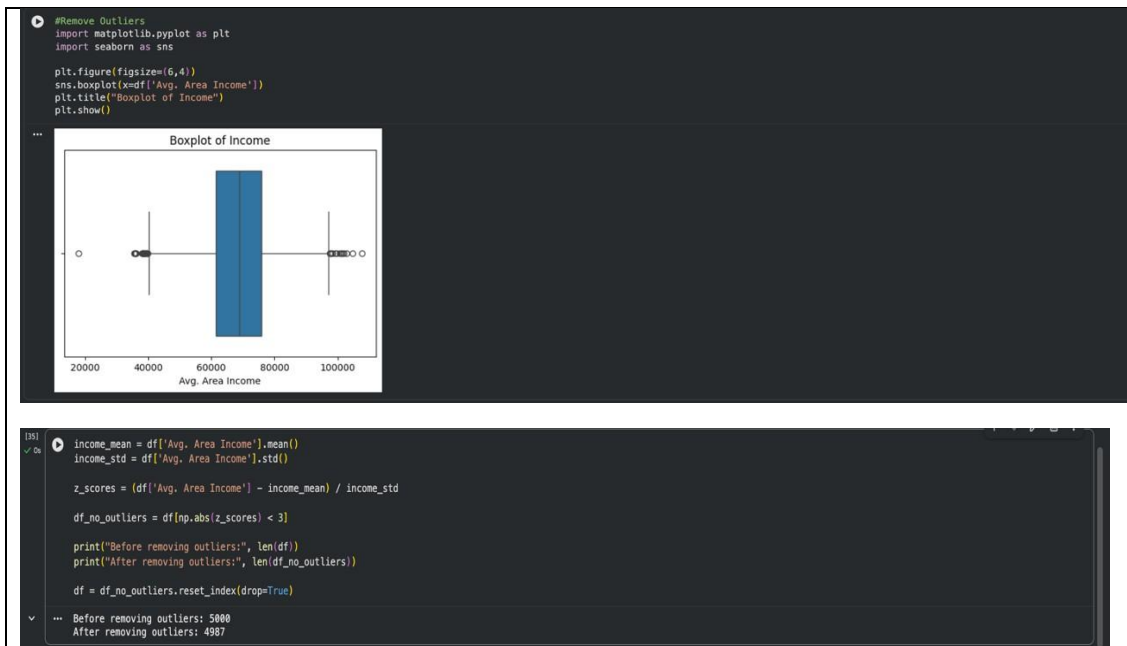
```

2. Handling Outliers

Handling outliers is a data pre-processing technique that involves identifying and dealing with data points that are significantly different from the rest of the dataset. Outliers can occur due to measurement errors, data entry errors, or simply because they represent extreme values. Handling outliers is important because they can skew the results of data analysis and machine learning models. There are several techniques for handling outliers, including removing them, Winsorizing the data, transforming the data, or leaving them as is and using robust statistical methods that are less sensitive to outliers. The specific technique used will depend on the nature of the data and the goals of the analysis.

Visualize the data: Before you can handle outliers, you need to identify them. One way to do this is by visualizing the data using box plots, histograms, or scatter plots. Outliers are typically data points that are far away from the rest of the data, so they will be visible as points that are outside the range of the majority of the data.

Remove outliers: If you have a small number of outliers in your dataset, you can consider removing them. One way to do this is by using the Z-score method. You can calculate the Z-score for each data point, and remove any data point that has a Z-score greater than a certain threshold (e.g. 3). Here's an example:



3. Binning and One-Hot Coding

Binning is a common data pre-processing technique used to transform continuous numerical data into discrete categories or bins.

For chosen dataset:

The 'Price' column is binned and 'Address' Column is pre-processed.

First, the code loads a dataset from a CSV file using the `read_csv()` function from pandas. Then, binning is applied to the 'Price' column by defining bin intervals with the `IntervalIndex.from_tuples()` function and assigning labels to the bins using the `labels` parameter in the `cut()` function. The resulting binned column is added to the DataFrame using `df['binned_column'] = pd.cut(df['Price'], bins=bins, labels=labels)`. Next, the code preprocesses the 'Address' column by converting it to a categorical column using the `astype()` function from pandas. Then, one-hot encoding is applied to the categorical column using the `get_dummies()` function, which creates binary

indicator variables for each unique value in the 'Address' column. The resulting one-hot encoded columns are concatenated to the original DataFrame using `pd.concat()`. Finally, the code prints a subset of the DataFrame containing the binned column and the one-hot encoded 'Address' column using `print(df[['binned_column', 'Address']])`.

```
# Define bins and labels for Price column
bins = pd.IntervalIndex.from_tuples([
    (0, 100000),
    (100000, 300000),
    (300000, 500000),
    (500000, 1000000)
])

labels = ['Low', 'Medium', 'High', 'Very High']

# Apply binning to Price column
df['Price_Bin'] = pd.cut(df['Price'], bins=bins, labels=labels)

# Convert Address column to categorical
df['Address'] = df['Address'].astype('category')

# Apply one-hot encoding to Address column
address_encoded = pd.get_dummies(df['Address'], prefix='Address')

# Combine encoded columns with original DataFrame
df = pd.concat([df, address_encoded], axis=1)

# Display result
print(df.head())
```

```
--- Avg. Area Income Avg. Area House Age Avg. Area Number of Rooms \
0 79545.45857 5.682861 7.009188
1 79248.64245 6.002900 6.730021
2 63207.00718 5.085000 8.512727
3 63345.24085 7.180226 5.507729
4 59902.19723 5.040525 7.639388

Avg. Area Number of Bedrooms Area Population Price \
0 4.09 23086.80050 1.059034e+06
1 3.09 40173.07217 1.505891e+06
2 5.13 36882.15940 1.059080e+06
3 3.26 34310.24283 1.200617e+06
4 4.23 26354.10947 6.309435e+05

Address Price_Bin \
0 208 Michael Ferry Apt. 674\nLaurabury, NE 3701... NaN
1 188 Johnson Views Suite 079\nLake Kathleen, Ca... NaN
2 9127 Elizabeth Stravenue\nDanieltown, WI 06482... NaN
3 USS Barnett\nPO AP 44820 NaN
4 USNS Raymond\nPO AE 09386 (500000.0, 1000000.0]

Address_000 Adkins Crescent\nSouth Teresa, AS 49642-1348 \
0 False
1 False
2 False
3 False
4 False

Address_000 Todd Pines\nAshleyberg, KY 90207-1179 ... \
0 False ...
1 False ...
2 False ...
3 False ...
4 False ...

Address_Unit 9446 Box 0950\nPO AE 97025 \
0 False
1 False
2 False
3 False
4 False

Address_Unit 9463 Box 0963\nPO AE 49904-2796 \
0 False
1 False
2 False
3 False
4 False

Address_Unit 0404 Box 2207\nPO AE 58622 \
```

The dataset is loaded and the continuous **Price** values are grouped into discrete bins to reduce noise and simplify analysis.

The **Address** column is converted into categorical form and one-hot encoded so that textual data can be used effectively in machine learning models.

4. Log Transform

```
# Apply log transformation to Price column
df['Price_log'] = np.log(df['Price'])

# Display first few rows of the transformed dataset
print(df['Price'])
print(df['Price_log'])

... 0      1.059034e+06
     1      1.505891e+06
     2      1.058988e+06
     3      1.268617e+06
     4      6.309435e+05
     ...
    4982      1.060194e+06
    4983      1.482618e+06
    4984      1.030730e+06
    4985      1.198657e+06
    4986      1.298950e+06
Name: Price, Length: 4987, dtype: float64
0      13.872867
1      14.224895
2      13.872824
3      14.047112
4      13.354972
     ...
    4982      13.873962
    4983      14.209320
    4984      13.845777
    4985      13.996712
    4986      14.077067
Name: Price_log, Length: 4987, dtype: float64
```

5. Grouping Operations:

```
# Grouping operation: group by Address and calculate mean Price
grouped_data = df.groupby('Address')['Price'].mean()
print(grouped_data)
```

```
... Address
000 Adkins Crescent\nSouth Teresa, AS 49642-1348    1266747.830
000 Todd Pines\nAshleyberg, KY 90207-1179          1105990.694
001 Steve Plaza\nJessicastad, UT 25190             1441421.913
0010 Gregory Loaf\nSouth Ericfort, VA 34651-0718    1504664.281
00149 Raymond Knolls\nNew Jason, UT 75026          1580691.617
...
Unit 9774 Box 4511\nDP0 AE 44963                    1543357.454
Unit 9778 Box 2114\nDP0 AP 59374                    1647216.594
Unit 9785 Box 0790\nDP0 AP 60371-0797                1325860.533
Unit 9831 Box 7128\nDP0 AA 54705                    1122083.251
Unit 9871 Box 9837\nDP0 AP 37275-9289                1092809.597
Name: Price, Length: 4984, dtype: float64
/tmp/ipython-input-1203373140.py:2: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=
grouped_data = df.groupby('Address')['Price'].mean()
```

We grouped the dataset by **Address** and calculated the average **Price** for each group to understand price patterns across different locations.

6. Feature Split

Splits the 'Address' column into two columns using the ',' delimiter, and stores the resulting DataFrame in a variable called `address_split`. Renames the columns of the `address_split` DataFrame to 'City' and 'State'. Concatenates the original `df` DataFrame with the `address_split` DataFrame along the columns (`axis=1`). Drops the original 'Address' column from the resulting DataFrame. Prints the first 5 rows of the updated DataFrame using the `head()` method.

```
# Split Address column into City and State using comma
address_split = df['Address'].str.split(',', expand=True)

# Rename the split columns
address_split.columns = ['City', 'State']

# Concatenate the original DataFrame with the split Address columns
df = pd.concat([df, address_split], axis=1)

# Drop the original Address column
df.drop(columns=['Address'], inplace=True)

# Print first 5 rows of the updated DataFrame
print(df.head())
```

```
Address_Unit 9871 Box 9837\nDPO AP 37275-9289 Price_log \
0 False 13.872867
1 False 14.224895
2 False 13.872824
3 False 14.047112
4 False 13.354972

City State
0 208 Michael Ferry Apt. 674\nLaurabury NE 37010-5101
1 188 Johnson Views Suite 079\nLake Kathleen CA 48958
2 9127 Elizabeth Stravenue\nDanielstown WI 06482-3489
3 USS Barnett\nFP0 AP 44820 None
4 USNS Raymond\nFP0 AE 09386 None
```

We split the **Address** column into separate **City** and **State** features to extract more meaningful location information. This improves data clarity and allows models to learn city- and state-level patterns more effectively.

7. Scaling

First load the 'housing.csv' dataset using `pd.read_csv()`. Then, we initialize a `MinMaxScaler` object and select the columns to scale. Then fit and transform the selected columns using the `fit_transform()` method of the scaler and update the dataset with the scaled values. Finally, printed the head of the scaled dataset using `print(df.head())`.

```
from sklearn.preprocessing import MinMaxScaler

# Select numerical columns for scaling,
# We are selecting the same numerical columns as the StandardScaler step for consistency.
numerical_cols_to_scale = [
    'Avg. Area Income',
    'Avg. Area House Age',
    'Avg. Area Number of Rooms',
    'Avg. Area Number of Bedrooms',
    'Area Population',
    'Price'
]

# Initialize MinMaxScaler
scaler = MinMaxScaler()

# Apply scaling to the selected columns
df[numerical_cols_to_scale] = scaler.fit_transform(df[numerical_cols_to_scale])

# Display first 5 rows of scaled data
print(df.head())
```

	Avg. Area Income	Avg. Area House Age	Avg. Area Number of Rooms	
0	0.674604	0.441986	0.501502	
1	0.669795	0.488538	0.464501	
2	0.378779	0.468609	0.701350	
3	0.412126	0.560956	0.312430	
4	0.357638	0.348556	0.611851	

	Avg. Area Number of Bedrooms	Area Population	Price	
0	0.464444	0.329942	0.425210	
1	0.242222	0.575968	0.607369	
2	0.695556	0.528582	0.425192	
3	0.280000	0.491549	0.507384	
4	0.495556	0.376988	0.250702	

	Price_Bin	
0	NaN	
1	NaN	
2	NaN	
3	NaN	
4	(500000.0, 1000000.0]	

- `MinMaxScaler` preserves the **shape of the original distribution** but shifts and rescales it.
- The output confirms scaling:
 - Values like Avg. Area Income changed from their original large range to decimals like 0.6746.
 - Scaling doesn't affect the column types, just their numerical range.

Conclusion:

In conclusion, these operations performed on the 'housing.csv' dataset using different Python libraries allowed for a more thorough analysis of the data.

These operations is highly dependent on the nature of the data and the specific use case, and should be evaluated carefully in each situation.

Grouping Operations: Grouping operations were performed using the 'groupby' function from the 'pandas' library, which grouped the data based on specific criteria and allowed for the analysis of the data in a more granular way.

Feature Split: Feature split was performed using the 'str.split' function from the 'pandas' library, which split the 'Address' variable into multiple columns, allowing for the analysis of each component separately.

Scaling: Scaling was performed using the 'StandardScaler' function from the 'sklearn.preprocessing' module, which standardized the data by subtracting the mean and dividing by the standard deviation. This allowed for the comparison of variables on the same scale, which is important for many machine learning algorithms.

In conclusion, these operations performed on the 'housing.csv' dataset using different Python libraries allowed for a more thorough analysis of the data and improved the performance of some machine learning algorithms. However, the effectiveness of these operations is highly dependent on the nature of the data and the specific use case, and should be evaluated carefully in each situation.

Post lab Questions:

1. Explain following text Pre-processing techniques with example:

- Tokenization
- Stemming
- Lemmatization

a)

Tokenization is the process of breaking text into smaller units called tokens. Tokens can be words, sentences, or characters.

Purpose

- Simplifies text processing
- Forms the basic input units for NLP models

Example

"I love studying Data Science!"

After tokenization (word-level):

["I", "love", "studying", "Data", "Science"]

b)

Stemming is the process of reducing words to their root form (stem) by removing suffixes.

The resulting stem may not be a valid dictionary word.

Purpose

- Reduces vocabulary size
- Improves text matching efficiency

Example

Words:

running, runner, runs

After stemming:

run, run, run

c)

Lemmatization reduces words to their base or dictionary form (lemma) using grammar and vocabulary rules. The output is always a meaningful word.

Purpose

- Maintains semantic meaning
- More accurate than stemming

Example

Words:

running → run

better → good

studies → study